

CONCEPT

What is a Parameterized Query?

A **parameterized query** (also called a *prepared statement*) separates the SQL structure from the data values. Instead of concatenating user input directly into SQL strings, you insert **placeholders** and supply the actual values separately.

Why it matters: When you build SQL by concatenating strings, an attacker can inject malicious SQL fragments. Parameterized queries make this impossible because the database engine treats parameter values as *data*, never as executable code.

Here's the dangerous way — raw string concatenation:

```
-- Python (UNSAFE!) query = f"SELECT * FROM users WHERE name =  
'{user_input}'" cursor.execute(query) -- If user_input is: ' OR 1=1 -- --  
The query becomes: SELECT * FROM users WHERE name = ' OR 1=1 --' -- This  
returns EVERY row!
```

BAD — VULNERABLE

Now the safe way — parameterized:

GOOD — SAFE

```
-- Python (SAFE) query = "SELECT * FROM users WHERE name = %s"
cursor.execute(query, (user_input,)) -- The database treats user_input as a
DATA value, -- never as part of the SQL command itself.
```

Placeholder syntax varies by platform: Python DB-API uses `%s`, SQLite uses `?`, PostgreSQL (psycopg2) supports both `%s` and named `%(name)s`, SQL Server / JDBC uses `?`, and many ORMs use `:name` style. In this tutorial we primarily use `%s` (positional) and `:name` (named) styles.

WORKED EXAMPLE

Your First Parameterized Query

Scenario: You have a table `patients` with columns `id`, `name`, `dob`, `diagnosis`. You need to look up a single patient by their `diagnosis` value that comes from user input.

```
-- Step 1: Write the query with a placeholder query = "SELECT id, name, dob
FROM patients WHERE diagnosis = %s" -- Step 2: Execute, passing the value in
a tuple cursor.execute(query, (user_diagnosis,)) -- Step 3: Fetch results
results = cursor.fetchall()
```

What happened:

- The `%s` placeholder tells the database "a value goes here."
- The second argument to `execute()` is a **tuple** containing the actual value. The database engine safely escapes and binds it.
- Even if `user_diagnosis` contains malicious SQL, it's treated as a plain string value.

PRACTICE

Fill in the Blanks

Each exercise presents a scenario and a query with missing pieces. Type the correct SQL or parameter syntax in each blank, then check your work. Use **Hint** if stuck, or **Reveal** to see the answer.

1

Simple SELECT with One Parameter

Easy

Retrieve all columns for employees in a specific department. The department name comes from user input.

employees (id, name, department, salary)

```
query = "SELECT * FROM employees
        WHERE department = %s"
cursor.execute(query, (department,))
```

Check Answer

Hint

Reveal

Reset

✓ All correct — nice work!

The `%s` placeholder stands in for the value. The second argument is a tuple with one element — note the trailing comma. The database safely binds the value at execution time.

2

Two Parameters in a WHERE Clause

Easy

Find orders for a specific `customer_id` that have a quantity greater than a threshold — both values from user input.

```
orders (id, customer_id, product, quantity, order_date)
```

```
query = "SELECT * FROM orders
        WHERE customer_id = %s
        AND quantity > %s"
cursor.execute(query, (customer_id, min_qty))
```

Check Answer

Hint

Reveal

Reset

✓ All correct — nice work!

Multiple placeholders map to the tuple values *in order*. The first `%s` binds to the first tuple element, the second `%s` to the second. Always match the count!

3 Named Parameters

Easy

Using **named parameter** style (`:name`), find products in a given category below a max price. This style is common in SQLAlchemy, Oracle, and others.

```
products (id, name, category, price)
```

```

query = "SELECT name, price FROM products
        WHERE category = :category
        AND price < :max_price"

cursor.execute(query, {"category": category, "max_price": max_price})

```

Check Answer

Hint

Reveal

Reset

✓ All correct — nice work!

Named parameters use `:key` in the SQL and a **dictionary** instead of a tuple. The keys in the dict must match the placeholder names. This is more readable when queries have many parameters.

4 INSERT with Parameters

Easy

Insert a new patient record. All three field values come from user input.

patients (id (auto), name, dob, diagnosis)

```

query = "INSERT INTO patients (name, dob, diagnosis)
        VALUES (:name, :dob, :diagnosis)"

cursor.execute(query, (name, dob, diagnosis))

```

[Check Answer](#)[Hint](#)[Reveal](#)[Reset](#)

✓ All correct — nice work!

Parameterization works with `INSERT` too. Each `%s` in the `VALUES` clause maps to one element in the tuple. This prevents injection through any of the input fields.

5

UPDATE with a WHERE Guard

Medium

Update the quantity for a specific sku in a given warehouse. All three values are user-supplied.

`inventory` (id, sku, product_name, quantity, warehouse)

```
query = "UPDATE inventory
        SET quantity = %s
        WHERE sku = %s
        AND warehouse = %s"
cursor.execute(query, (
    quantity, sku, warehouse
))
```

[Check Answer](#)[Hint](#)[Reveal](#)[Reset](#)

✓ All correct — nice work!

Parameterized **UPDATE** queries follow the same pattern. The order in the tuple matches the *order of placeholders in the SQL*, not the clause order. Here: SET value first, then the two WHERE values.

6 LIKE Pattern with Parameter

Medium

Search for articles whose title contains a keyword (partial match). The keyword comes from user input. Remember: the % wildcards are part of the *value*, not the SQL.

```
articles (id, title, body, author, published_date)
```

```
search_term = " " + keyword + " "
query = "SELECT title, author FROM articles
WHERE title LIKE %S"
cursor.execute(query, (search_term,))
```

Check Answer

Hint

Reveal

Reset

Answer revealed. Study it, then try the next one from memory!

With LIKE, the wildcards % must be part of the *parameter value*, not hard-coded in the SQL string. Build the pattern in application code first, then pass the complete pattern as a single parameter.

7 DELETE with Named Parameters

Medium

Delete expired sessions for a specific user. Use **named parameters**. The user ID and a cutoff timestamp are inputs.

```
sessions (id, user_id, token, created_at, expires_at)
```

```
query = "DELETE FROM sessions
        WHERE user_id = :uid
        AND expires_at < :cutoff"
cursor.execute(query, {
    "uid": uid,
    "cutoff": cutoff
})
```

Check Answer

Hint

Reveal

Reset

✓ All correct — nice work!

Named parameters in **DELETE** statements work identically. The **:uid** and **:cutoff** names must match the dictionary keys exactly.

8

Parameterized JOIN Query

Medium

Join orders with customers to find orders above a minimum total for customers in a specific city.

```
orders(id, customer_id, total) & customers(id, name, city)
```



```

query = """
    SELECT c.name, o.id, o.total
    FROM orders o
    JOIN
        join keyword      customers c ON o.customer_id = c.id
    WHERE c.city =
        placeholder 1
    AND o.total >
        placeholder 2
    """

cursor.execute(query, (
    city,
    min_total,
    value 1,
    value 2
))

```

Check Answer

Hint

Reveal

Reset

✓ All correct — nice work!

Parameters work in JOINS the same as simple queries. Note that the `JOIN ... ON` clause uses *column references* (not parameterized) — only **data values** should be parameterized, never table/column names.

9 IN Clause with Dynamic List

Hard

Select products whose category is in a user-provided list of categories. You must dynamically build the right number of placeholders.

products (id, name, category, price, in_stock)

```

categories = ["Electronics", "Books", "Toys"]

placeholders = ", ".join(["%s"] * len(categories))

query = f"SELECT * FROM products
        WHERE category IN ({placeholders})"

cursor.execute(query, categories)

```

Check Answer

Hint

Reveal

Reset

✓ All correct — nice work!

You cannot pass a list to a single `%s`. Instead, generate one `%s` per item and join them with commas. The f-string inserts this placeholder string into the SQL, then the list is passed as the parameter sequence. This stays injection-safe because each element is individually bound.

10 Parameterized Subquery with Aggregation

Hard

Find sales reps whose total sales in a given region exceed an amount threshold, after a specific start date. Uses a subquery with GROUP BY and HAVING — all three filter values are parameterized.

```

sales(id, rep_id, amount, region, sale_date) & reps(id, name, hire_date)

```

```

query = """
    SELECT r.name, sub.total_sales
    FROM reps r
    JOIN (

```

```

SELECT rep_id, SUM(amount) AS total_sales
FROM sales
WHERE region = %s
AND sale_date >= %s
GROUP BY rep_id
HAVING SUM(amount) > %s
) sub ON r.id = sub.rep_id
ORDER BY sub.total_sales DESC
"""
cursor.execute(query, (
    region,
    start_date,
    min_amount
))

```

The code above shows a SQL query with three placeholders (%s) and three corresponding values (value 1, value 2, value 3) passed to the cursor.execute() method. The placeholders are used in the WHERE, AND, and HAVING clauses. The values are passed as a tuple to the cursor.execute() method.

Check Answer

Hint

Reveal

Reset

✓ All correct — nice work!

Even inside subqueries, parameterization works identically. The three %s placeholders bind to the tuple in order. The HAVING clause filters *after* aggregation — it's the GROUP BY equivalent of WHERE. This pattern shows that arbitrarily complex SQL still uses the same safe binding mechanism.